

Programação Dinâmica

Motivação

- Técnica utilizada em problemas de otimização
- Da mesma forma que o backtrack recursivo
 - Baseia-se em uma recorrência
 - Mas preenche uma tabela ao invés de fazer chamadas recursivas
- Mais eficiente que o backtrack recursivo quando
 - uma mesma subinstância ocorre várias vezes na árvore de recursão,
 - pois a PD resolve cada subinstância apenas uma vez.
- PD fornece algoritmo polinomial para vários problemas importantes
- Uma forma de projetar é
 - Construa um algoritmo de backtrack recursivo ou defina sua relação de recorrência
 - Mecanicamente adapte para um algoritmo de programação dinâmica

Passos para projetar backtrack recursivo

1. Especificação
2. Construção das subinstâncias
3. O que é recebido das chamadas recursivas?
4. Construção da solução (utilizando o retorno de cada chamada recursiva)
5. Cálculo da medida de sucesso desta solução
6. Como determinar a melhor decisão (dentre as soluções construídas)?
7. Caso base
8. Pseudocódigo
9. Recorrência

Passos para projetar backtrack recursivo

algoritmo **BacktrackRecursivo**(I)

<pré-cond>: I é uma instância do problema

<pós-cond>: retorna uma solução ótima para I, e seu custo (minimização)

se I é suficiente pequena (caso base)

 retorne a solução e custo deste caso base

senão

 optCusto = ∞

 para cada decisão k possível

 Ik = instância que resulta de I após a decisão k

 sol_k, custo_k = **BacktrackRecursivo**(Ik)

 sol, custo = solução e custo p/ I usando decisão k (sol_k e custo_k)

 se custo < optCusto

 optSol = sol

 optCusto = custo

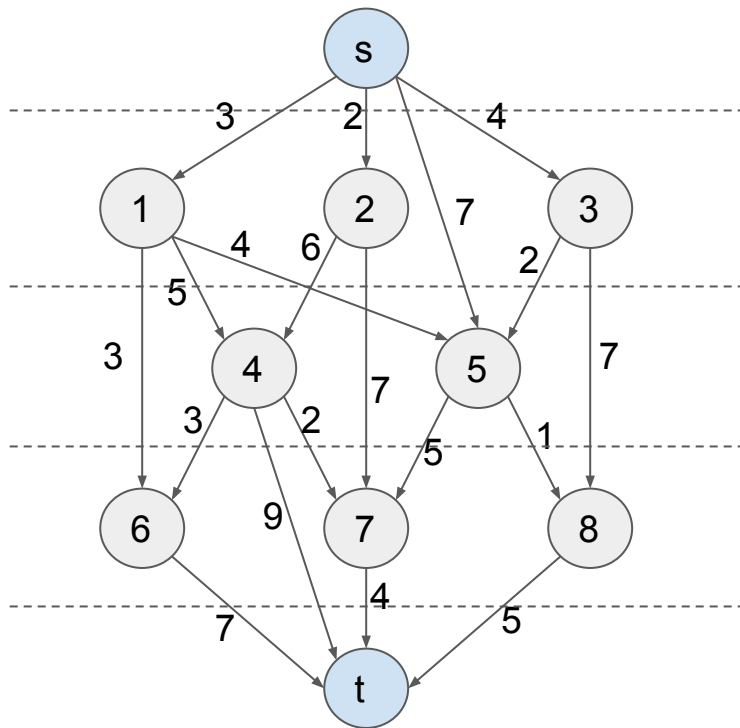
 retorne optSol, optCusto

Ex.: caminho mais curto em grafos em camadas

Grafo direcionado com pesos nas arestas

Nós são organizados em camadas

Toda aresta vai de camada mais alta para mais baixa



Encontrar caminho mais curto de **s** até **t**

Se os nós forem numerados da camada mais alta para mais baixa...

Toda aresta vai de número menor p/ maior

Ex.: caminho mais curto em grafos em camadas

- Especificação

- Instâncias

- Grafo direcionado G em camadas e com pesos nas arestas

- Nós são numerados de 0 até n
 - Para toda aresta (i,j) em G , temos que $i < j$
 - O peso da aresta (i,j) é denotado por $w[i,j]$
 - Obs.: grafo em camadas não possui ciclos

- Dois nós s e t de G , onde s é o nó 0 e t é o nó n .

- Soluções: caminho de s até t em G

- Custo da solução: soma dos pesos das arestas no caminho

- Objetivo: minimizar o custo

- Força bruta: existe quantidade exponencial (em n) de caminhos entre s e t

Ex.: caminho mais curto em grafos em camadas

- Construção das subinstâncias

- Instância (G, u, t) : grafo G , nó origem u , nó destino t
- Decisão: para qual vizinho v de u seguir para obter um caminho mais curto de u até t ?
- Para cada vizinho v de u , desejamos encontrar o caminho mais curto de v até t
- Subinstância:
 - Possui o mesmo grafo G e o mesmo destino t
 - Porém, inicia no nó v
- Subinstância é menor?
 - Tamanho: número de camadas de v até t
 - A camada de v é mais próxima de t que a camada de u

- O que é recebido das chamadas recursivas?

- Solução ótima da subinstância: um caminho mais curto de v até t
- Custo desta solução: soma dos pesos das arestas

Ex.: caminho mais curto em grafos em camadas

- Construção da solução e cálculo do custo
 - Estamos construindo a solução para a instância (G, u, t)
 - Para cada vizinho v de u
 - A chamada recursiva retorna um caminho mais curto optSubSol de v até t
 - E retorna também o custo optSubCusto desta solução
 - Então $\text{optSubSol} \cup \{(u, v)\}$ é um caminho mais curto de u até t passando por v
 - Com custo $\text{optSubCusto} + w[u, v]$
- Qual a melhor decisão?
 - Seguimos pelo vizinho v que minimiza $\text{optSubCusto} + w[u, v]$
- Caso base
 - Quando o nó de origem u é igual ao nó de destino t
 - Neste caso o caminho é vazio e tem custo zero

Ex.: caminho mais curto em grafos em camadas

- Pseudocódigo

algoritmo **CamMaisCurtoGrafoCamadas** (G, u, t)

<pré-cond>: G é grafo direcionado em camadas com pesos $w[]$ nas arestas, u e t nós de G

<pós-cond>: retorna caminho mais curto de u até t , e seu custo.

se $u == t$

 retorne $(\{\}, 0)$

senão

$optCusto = \infty$

 para cada vizinho v de u em G

$(optSubSol, optSubCusto) = \text{CamMaisCurtoGrafoCamadas}(G, v, t)$

 se $optSubCusto + w[u, v] < optCusto$

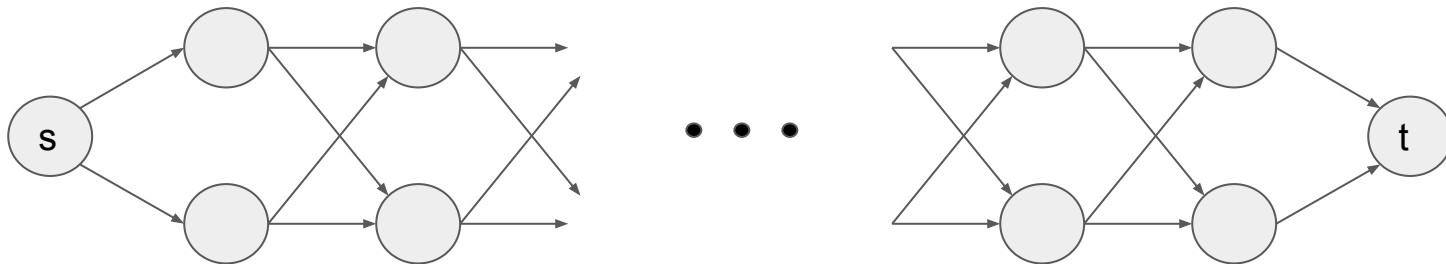
$optSol = optSubSol \cup \{(u, v)\}$

$optCusto = optSubCusto + w[u, v]$

 retorne $(optSol, optCusto)$

Ex.: caminho mais curto em grafos em camadas

- Recorrência para o custo da solução ótima
 - $\text{Custo}[G,u,t] = \min_{v \text{ vizinho de } u} (\text{Custo}[G,v,t] + w[u,v])$
 $\text{Custo}[G,t,t] = 0$
- Tempo de execução: exponencial, pois enumera todos os caminhos



k camadas intermediárias, $2k+2$ nós, $4k$ arestas, 2^k caminhos possíveis.

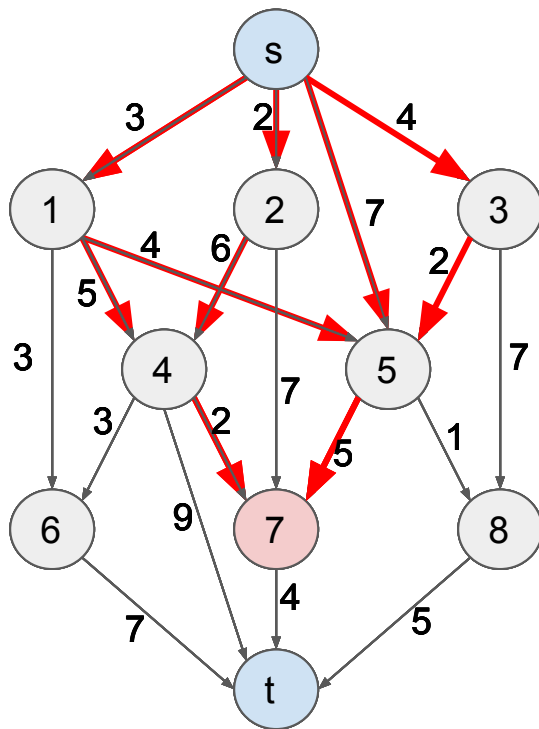
Passos para construir algoritmo de prog. dinâmica

0. Construa algoritmo de backtrack recursivo, ou relação de recorrência
1. Determine conjunto de subinstâncias
 - Todas as subinstâncias que são passadas nas chamadas do backtrack recursivo
 - Subinstâncias adicionais podem ocorrer (para simplificar), mas devem ser evitadas
 - Ex. (grafo camadas): (G, u, t) para todo nó u de G .
 - Note que as subinstâncias (G, u, v) , para $v \neq t$, são excluídas
 - PD é mais eficiente que o BR quando o BR resolve a mesma subinstância várias vezes
 - Pois a PD resolve cada subinstância apenas uma vez

Passos para construir algoritmo de prog. dinâmica

Redundância:

A subinstância $(G, 7, t)$ ocorre em vários ramos da árvore de recursão.



Portanto, PD é mais eficiente que BR para este problema.

Passos para construir algoritmo de prog. dinâmica

2. Determine o número de subinstâncias

- Função do tamanho da entrada
- Tem influência na complexidade da programação dinâmica
- Pode ser necessário reformular o backtrack recursivo para reduzir o número de subinstâncias
- Ex. (grafo camadas): $\{(G,u,t)$, para todo nó $u\}$, então o tamanho é o número de nós

3. Construa uma tabela indexada pelas subinstâncias

- Cada elemento armazena a solução ótima da subinstância e seu valor/custo
- Geralmente usamos duas tabelas: uma para a solução ótima e outra para seu valor/custo
- Ex. (grafo camadas):
 - $\text{optSol}[u]$: solução ótima da subinstância (G,u,t)
 - $\text{optCusto}[u]$: custo da solução ótima da subinstância (G,u,t)

Passos para construir algoritmo de prog. dinâmica

4. Construa a solução usando as soluções de outras subinstâncias

- Isto é feito de forma idêntica ao backtrack recursivo
 - A diferença é que a PD acessa a tabela, ao invés de fazer chamadas recursivas
- Ex. (grafo camadas): para cada vizinho v de u , se $\text{optCusto}[v] + w[u,v] < \text{optCusto}[u]$
 - $\text{optSol}[u] = \text{optSol}[v] \cup \{(u,v)\}$
 - $\text{optCusto}[u] = \text{optCusto}[v] + w[u,v]$

5. Casos base

- Mesmas subinstâncias que formam o caso base no backtrack recursivo
- PD inicializa as tabelas para as subinstâncias do caso base
- Ex. (grafo camadas): caso base ocorre quando $u == t$
 - $\text{optSol}[t] = \{\}$
 - $\text{optCusto}[t] = 0$

Passos para construir algoritmo de prog. dinâmica

6. Defina a ordem de preenchimento da tabela

- Ao calcular a solução de uma subinstância, todas as subinstâncias necessárias já devem estar preenchidas na tabela
- Pode usar qualquer ordem que respeite a dependência entre as subinstâncias
 - Ou qualquer ordem que vá das instâncias menores para as maiores
- Ex. (grafo camadas):
 - Como toda aresta vai de número de nó menor para maior,
 - preencha em ordem decrescente de número do nó.
- PD é um algoritmo iterativo onde o invariante do loop significa:
 - Toda subinstância necessária para resolver a subinstância da iteração já foi preenchida.

Passos para construir algoritmo de prog. dinâmica

7. Solução final

- Solução da instância original é a última subinstância preenchida na tabela (maior subinstância)
- Basta esta última solução preenchida, e seu valor/custo
- Ex. (grafo camadas): Nó de número 0 é o último preenchido.
 - Então retornamos `optSol[0]` e `optCusto[0]`.
- Para reduzir o armazenamento,
 - `optSol[u]` pode conter apenas a decisão tomada na subinstância `u`
 - Isto permite recuperar a sequência de decisões tomadas na solução retornada

Passos para construir algoritmo de prog. dinâmica

8. Pseudocódigo

algoritmo GrafoCamadasPD(G, s, t)

<pré-cond>: G é grafo em camadas com pesos $w[]$ nas arestas, s e t dois nós de G .

Nós são numerados de 0 até n , sendo $s == 0$ e $t == n$.

Para toda aresta (u, v) , $u < v$.

<pós-cond>: retorna arestas em um cam. mais curto de s até t , e as somas de seus pesos

$optSol[0..n]$ é uma lista de conjuntos

$optCusto[0..n]$ é uma lista de números, inicializados com oo

$optSol[n] = \{\}$; $optCusto[n] = 0$

para i de $n-1$ até 0

 // Cálculo similar ao backtrack recursivo

 para cada j vizinho de i

 se $optCusto[j] + w[i, j] < optCusto[i]$

$optSol[i] = optSol[j] \cup \{(u, v)\}$

$optCusto[i] = optCusto[j] + w[i, j]$

retorne ($optSol[0]$, $optCusto[0]$)

Passos para construir algoritmo de prog. dinâmica

9. Tempo de execução

- Tamanho da tabela (núm. subinstâncias) x tempo para preencher um elemento (solução)
- Ex. (grafo camadas):
 - Tabela tem $\theta(n)$ elementos
 - Em cada subinstância percorremos $O(n)$ vizinhos
 - Total: $O(n^2)$

Ex.: distância de edição

- Dadas duas strings $A[1..m]$ e $B[1..n]$, determine o número mínimo de edições de caracteres (inclusão, remoção ou troca) para igualá-las.

Exemplo: transformar “snowy” em “sunny”

S - N O W Y

S U N N - Y

= i = t r =

custo = 3

- S N O W - Y

S U N - - N Y

i t = r r i =

custo = 5

Ex.: distância de edição

- Seja $D(i,j)$ a distância de edição de $A[1..i]$ e $B[1..j]$.

A: S N O W **Y**

B: S U N N **Y**

$D(5,5) = D(4,4)$

A: S N O **W**

B: S U N **N**

$D(4,4) = 1 + D(3,3)$ (troca)

A: S N O W

B: S U N **N**

$D(4,4) = 1 + D(4,3)$ (N foi inserido)

A: S N O **W**

B: S U N N

$D(4,4) = 1 + D(3,4)$ (W foi removido)

Caso base:

- Se A é vazia, insira todos de B.
- Se B é vazia, remova todos de A.

Ex.: distância de edição

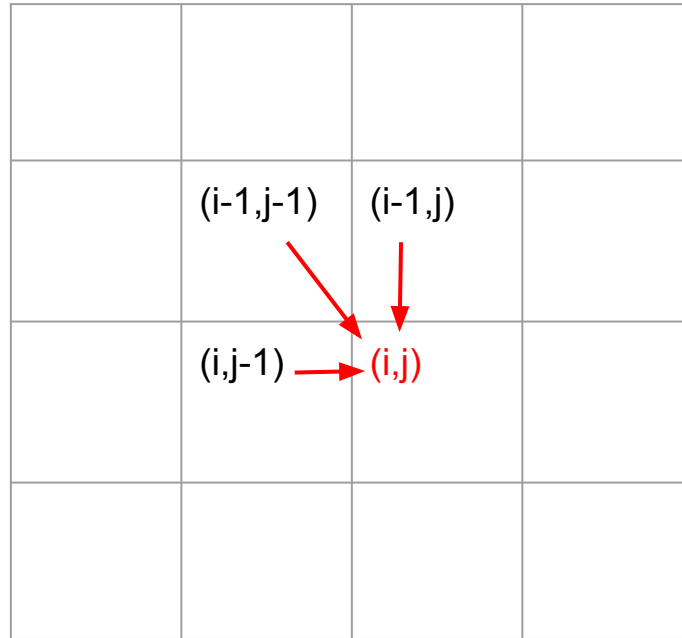
Recorrência:

$$\begin{aligned} D(i, j) &= i, && \text{se } j = 0 \\ D(i, j) &= j, && \text{se } i = 0 \\ D(i, j) &= D(i - 1, j - 1), && \text{se } A[i] = B[j] \\ D(i, j) &= 1 + \min(\underbrace{D(i - 1, j - 1)}_{\text{troca}}, \underbrace{D(i, j - 1)}_{\text{inserção}}, \underbrace{D(i - 1, j)}_{\text{remoção}}), && \text{se } A[i] \neq B[j] \end{aligned}$$

Ex.: distância de edição

```
algoritmo DistânciaEdição(A[1..m], B[1..n], i, j):  
    <pré-cond>: A e B são strings e i e j são inteiros.  $1 \leq i \leq m$ ,  $1 \leq j \leq n$ .  
    <pós-cond>: menor número de edições para transformar A[1..i] em B[1..j].  
    se i == 0  
        retorne j  
    senão, se j == 0  
        retorne i  
    senão  
        se A[i] == B[j]  
            retorne DistânciaEdição(A,B, i-1, j-1)  
        senão  
            retorne 1 + min( DistânciaEdição(A,B, i-1, j-1), // troca  
                             DistânciaEdição(A,B, i,   j-1), // inclusão  
                             DistânciaEdição(A,B, i-1, j  ) ) // remoção
```

Ex.: distância de edição



Ex.: distância de edição

arritmo **DistânciaEdiçãoPD**(A[1..m], B[1..n]):

<pré-cond>: A e B são strings de tamanho n e m, respectivamente

<pós-cond>: menor número de edições (inserção, remoção, troca) para transformar A em B

D[0..m, 0..n] = matriz de inteiros

para i de 0 até m

 D[i,0] = i

para j de 1 até n

 D[0,j] = j

para i de 1 até m

 para j de 1 até n

 se A[i] == B[j]

 D[i,j] = D[i-1,j-1]

 senão

 D[i,j] = 1 + min(D[i-1,i-1],

 D[i,j-1],

 D[i-1,j])

retorne D[m,n]

Tamanho da tabela: $\theta(mn)$

Cada elemento: $\theta(1)$

Complexidade de tempo: $\theta(mn)$

[illegible]

[illegible]

[illegible]

[illegible]

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

		P	O	L	I	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	EXPONENTIAL POLYNOMIAL rr==tt==i==					10
P	3	2	3	3	4						10
O	4	3	2	3	4						9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
C	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	6	6	6	6	6	7	8
A	10	9	8	8	7	7	7	7	7	6	7
L	11	10	9	8	8	8	8	8	8	7	6

Subinstâncias comuns em programação dinâmica

1. A entrada é $x_1, x_2, \dots, x_n$
um subproblema é $x_1, x_2, \dots, x_i$ para todo $1 \leq i \leq n$

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \quad x_6 \quad x_7 \quad x_8 \quad x_9 \quad x_{10}$

- Ex.: caminho mínimo em grafo em camadas (i primeiros nós)
- Neste caso o número de subproblemas é $\theta(n)$

Subinstâncias comuns em programação dinâmica

2. A entrada é $x_1, x_2, \dots, x_n$
e um subproblema é $x_i, x_{i+1}, \dots, x_j$ para todo par (i,j) com $1 \leq i \leq j \leq n$

x_1 x_2 x_3 x_4 x_5 x_6 x_7 x_8 x_9 x_{10}

- Neste caso o número de subproblemas $\Theta(n^2)$

$$\sum_{i=1}^n \sum_{j=i}^n 1 = \sum_{i=1}^n (n - i + 1) = \sum_{k=1}^n k \in \Theta(n^2)$$

Subinstâncias comuns em programação dinâmica

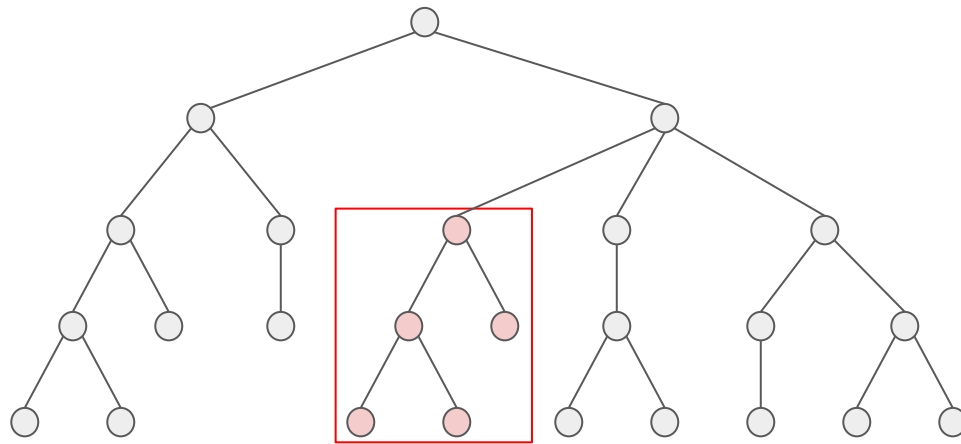
3. A entrada é $x_1, x_2, \dots, x_n$ e $y_1, y_2, \dots, y_m$
um subproblema é $x_1, x_2, \dots, x_i$ e $y_1, y_2, \dots, y_j$ para todo $1 \leq i \leq n$ e $1 \leq j \leq m$

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8		

- Ex.: distância de edição (primeiros i caracteres de A, e primeiros j caracteres de B)
- Neste caso o número de subproblemas é $\theta(nm)$

Subinstâncias comuns em programação dinâmica

4. A entrada é uma árvore enraizada T , e um subproblema é uma subárvore de T contendo um nó u e todos seus descendentes



- Neste caso o número de subproblemas é $\theta(n)$, onde n é o número de nós na árvore T
 - Cada nó é raiz de uma subárvore